

Lecture Notes

Chapter 5

Regularization & Evaluation

Topics Covered:

- 5.1 Two Failure Cases of Supervised Learning (Overfitting, Underfitting)
- 5.2 Why Model Evaluation Matters
- 5.3 Dataset Splitting (Train/Validation/Test, Holdout)
- 5.4 Cross-Validation Methods
- 5.5 Bootstrap Method
- 5.6 Evaluation Metrics for Classification
- 5.7 Evaluation Metrics for Regression
- 5.8 Regularization: Motivation and Intuition
- 5.9 L2 Regularization
- 5.10 L1 Regularization
- 5.11 Early Stopping
- 5.12 Ensemble Methods Overview
- 5.13 Bagging and Random Forests
- 5.14 Boosting Methods

Prepared by:

Sharifur Rahman

Lecturer, Department of CSE
Green University of Bangladesh

March 9, 2026

Contents

1	Two Failure Cases of Supervised Learning	3
1.1	Underfitting	3
1.2	Overfitting	3
1.3	The Ideal: Good Fit	4
1.4	Bias–Variance Trade-off	4
2	Why Model Evaluation Matters	6
2.1	The Core Problem	6
2.2	What We Really Want	6
2.3	Evaluation Tells Us	6
3	Dataset Splitting	7
3.1	Train / Validation / Test Split	7
3.2	Holdout Method	7
4	Cross-Validation Methods	8
4.1	K-Fold Cross-Validation	8
4.2	Leave-One-Out Cross-Validation (LOOCV)	9
4.3	Stratified K-Fold	9
5	Bootstrap Method	10
5.1	How It Works	10
6	Evaluation Metrics for Classification	11
6.1	Confusion Matrix	11
6.2	Key Metrics	11
6.2.1	Accuracy	11
6.2.2	Precision	12
6.2.3	Recall (Sensitivity)	12
6.2.4	F1-Score	12
6.3	ROC Curve and AUC	12
7	Evaluation Metrics for Regression	14
7.1	Mean Absolute Error (MAE)	14
7.2	Mean Squared Error (MSE)	14
7.3	Root Mean Squared Error (RMSE)	14
7.4	R-Squared (R^2)	14
8	Regularization: Motivation and Intuition	16
8.1	The Overfitting Problem Revisited	16
8.2	Intuition: Penalty on Complexity	16
9	L2 Regularization (Ridge Regression)	17
9.1	How It Works	17
9.2	Geometric Interpretation	17
9.3	Closed-Form Solution	17

10 L1 Regularization (Lasso)	19
10.1 Key Property: Sparsity	19
10.2 Geometric Interpretation	19
10.3 L1 vs L2 Comparison	20
11 Early Stopping	21
11.1 The Key Idea	21
11.2 Algorithm	21
12 Ensemble Methods Overview	23
12.1 Why Combine Models?	23
12.2 Main Ensemble Strategies	23
13 Bagging and Random Forests	24
13.1 Bagging (Bootstrap Aggregating)	24
13.2 Random Forests	24
13.3 Key Hyperparameters of Random Forests	25
14 Boosting Methods	26
14.1 AdaBoost (Adaptive Boosting)	26
14.2 Gradient Boosting	27
14.3 XGBoost	27
14.4 Boosting vs Bagging Comparison	27

1 Two Failure Cases of Supervised Learning

When we train a machine learning model, two common problems can occur: **Overfitting** and **Underfitting**. Understanding these problems helps us build better models.

1.1 Underfitting

Definition

Underfitting happens when a model is *too simple* to capture the patterns in the data. The model performs poorly on both the training data *and* new (unseen) data.

Why does it happen?

- The model has too few parameters (e.g. a straight line for curved data).
- The model is not trained long enough.
- Important features are missing from the input.

Example

Imagine students' exam scores depend on study hours in a curved way. If we fit a *straight line* to this curved data, the line cannot capture the curve—this is underfitting.

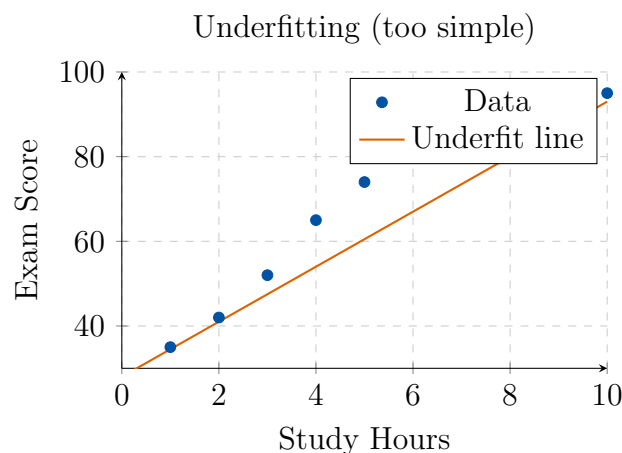


Figure 1: Underfitting: a straight line misses the curved pattern in the data.

1.2 Overfitting

Definition

Overfitting happens when a model is *too complex* and learns the training data *too well*—including its noise and random fluctuations. The model works well on training data but fails on new data.

Why does it happen?

- The model has too many parameters relative to the data size.
- Training data is too small.
- The model is trained for too many iterations.

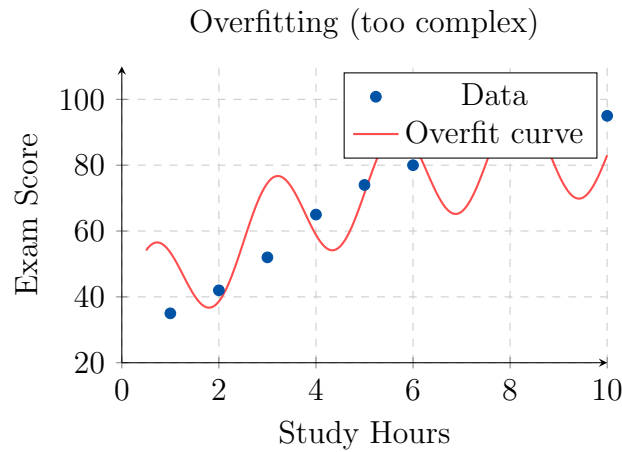


Figure 2: Overfitting: the model memorises noise and fits training data too tightly.

1.3 The Ideal: Good Fit

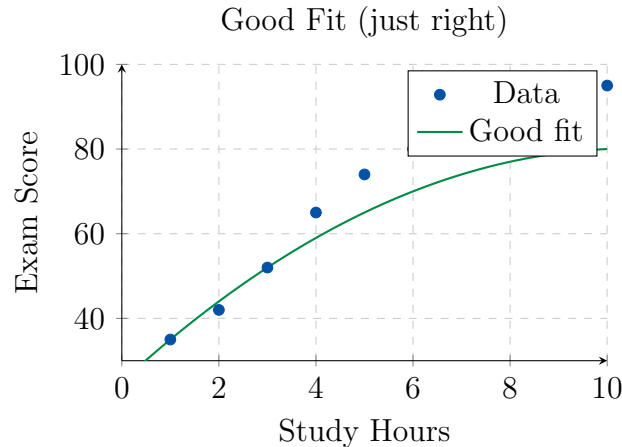


Figure 3: A good fit captures the underlying pattern without memorising noise.

1.4 Bias–Variance Trade-off

These two problems are related to two sources of error:

Problem	Bias	Variance
Underfitting	High (wrong assumptions)	Low
Overfitting	Low	High (sensitive to noise)
Good model	Low	Low

Key Insight

The goal of machine learning is to find the **sweet spot** between bias and variance so the model generalises well to new data.

2 Why Model Evaluation Matters

Definition

Model evaluation is the process of measuring how well a trained model performs on data it has never seen before (unseen/test data).

2.1 The Core Problem

If we only measure performance on training data, we get a misleading picture. A model can score 100% on training data (by memorising it) but fail completely on real-world data.

Example

A student memorises every answer in a practice test. They score 100% on that practice test. But in the actual exam with new questions, they score 30%. The practice score is *not* a reliable measure of true ability. The same thing happens with machine learning models.

2.2 What We Really Want

We want a model that:

1. Learns general patterns from training data.
2. Works well on **new, unseen examples** (generalisation).

2.3 Evaluation Tells Us

- Is the model underfitting or overfitting?
- Which of several models is best?
- Is the model ready to deploy in the real world?
- How to tune hyperparameters?

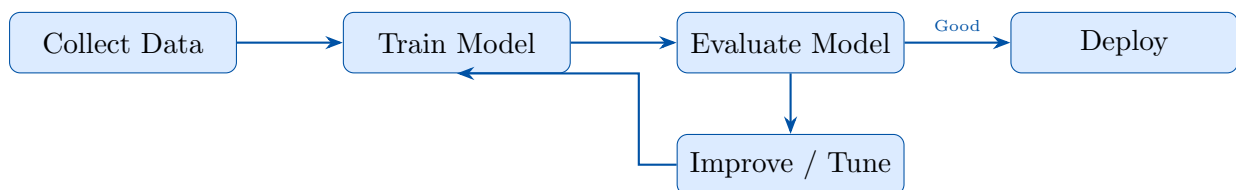


Figure 4: The role of evaluation in the machine learning workflow.

3 Dataset Splitting

To evaluate a model fairly, we split our dataset into separate parts.

3.1 Train / Validation / Test Split

Definition

- **Training set** – Used to *fit* (train) the model.
- **Validation set** – Used to *tune* hyperparameters and select the best model during development.
- **Test set** – Used *only once* at the end to report the final performance. It simulates unseen real-world data.

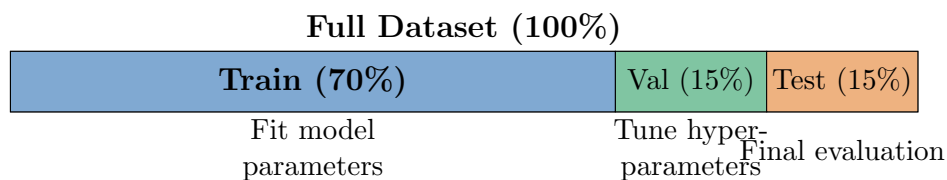


Figure 5: A typical 70/15/15 train/validation/test split.

3.2 Holdout Method

The **holdout method** is the simplest form of dataset splitting. We split the data into only two parts: training and test (no separate validation set).

Example

Suppose we have 1000 labelled emails (spam / not spam).

- **Training set:** 800 emails – the model learns from these.
- **Test set:** 200 emails – we evaluate the model on these.

The model never sees the test set during training.

Rule of thumb for splitting:

Dataset Size	Suggested Split
Small (< 1000)	80% train / 20% test
Medium (1000–100 000)	70% train / 15% val / 15% test
Large (> 100 000)	98% train / 1% val / 1% test

Note

Always **shuffle** the data randomly before splitting (unless the data has a time order, such as stock prices). This prevents biased splits.

4 Cross-Validation Methods

The holdout method has a problem: the result depends heavily on *which* samples land in the test set. Cross-validation solves this by using the data more efficiently.

4.1 K-Fold Cross-Validation

Definition

In **K-Fold Cross-Validation**, the data is divided into K equal *folds* (parts). The model is trained K times. Each time, one fold is used as the test set and the remaining $K - 1$ folds are used for training. The final score is the *average* of all K scores.

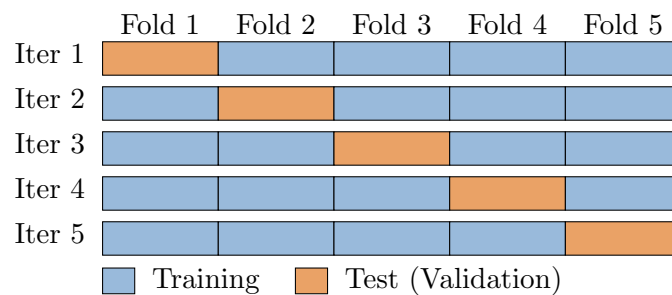


Figure 6: 5-Fold Cross-Validation. Each fold acts as the test set exactly once.

Algorithm:

1. Shuffle the dataset randomly.
2. Split into K equal folds.
3. **For** $i = 1$ to K :
 - Use fold i as the test set.
 - Train on the remaining $K - 1$ folds.
 - Record accuracy (or other metric) A_i .
4. Final score = $\frac{1}{K} \sum_{i=1}^K A_i$

Example

We have 500 data points and $K = 5$. Each fold has 100 points.

- Iteration 1: test = fold 1, train = folds 2,3,4,5 $\Rightarrow$ Accuracy = 88%
- Iteration 2: test = fold 2, train = folds 1,3,4,5 $\Rightarrow$ Accuracy = 90%
- ... (repeat for all 5 folds)
- Average accuracy = $(88 + 90 + 87 + 91 + 89)/5 = 89\%$

4.2 Leave-One-Out Cross-Validation (LOOCV)

Definition

LOOCV is a special case of K-Fold where $K = N$ (the total number of samples). In each iteration, exactly *one* sample is used as the test set.

- **Advantage:** Uses almost all data for training each time.
- **Disadvantage:** Very slow for large datasets (requires N training runs).

4.3 Stratified K-Fold

Definition

Stratified K-Fold ensures that each fold has the *same proportion* of each class as the full dataset. This is important when classes are imbalanced.

Example

If 80% of emails are not-spam and 20% are spam, stratified K-Fold ensures each fold also has roughly 80% / 20% split.

5 Bootstrap Method

Definition

The **Bootstrap** method estimates model performance by creating many *random samples with replacement* from the original dataset.

5.1 How It Works

Suppose our dataset has N samples.

1. Randomly draw N samples **with replacement** to create a bootstrap sample B . (Some original samples appear multiple times; some do not appear at all.)
2. Train the model on B .
3. Test the model on the samples *not* in B (called the **out-of-bag** samples).
4. Repeat steps 1–3 many times (e.g. 100 or 1000 times).
5. Average the out-of-bag scores.

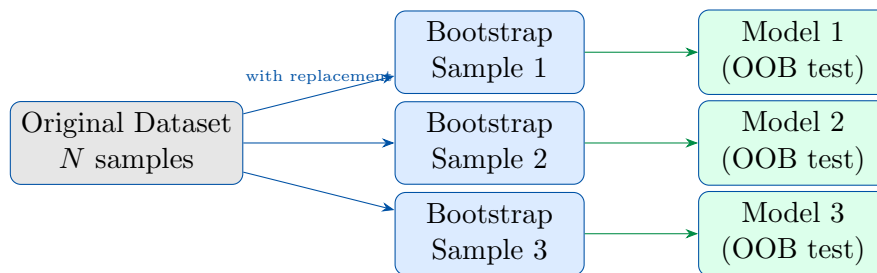


Figure 7: Bootstrap sampling: each sample is drawn with replacement and evaluated on out-of-bag data.

Key Insight

On average, each bootstrap sample contains about **63.2%** of the original data points (the rest are duplicates). About 36.8% are left out as out-of-bag samples.

Example

Original data: {A, B, C, D, E} (5 samples)
Bootstrap sample (drawn with replacement): {A, C, C, B, E}
Out-of-bag (not selected): {D}
⇒ Train on {A, B, C, E}, test on {D}.

6 Evaluation Metrics for Classification

After training a classifier, we need numbers to measure how good it is.

6.1 Confusion Matrix

Definition

A **confusion matrix** is a table that summarises the predictions of a classifier against the true labels.

		Actual Class	
		Positive	Negative
Predicted Class	Positive	✓ TP True Positive Predicted: Positive Actual: Positive	✗ FP False Positive Predicted: Positive Actual: Negative
	Negative	✗ FN False Negative Predicted: Negative Actual: Positive	✓ TN True Negative Predicted: Negative Actual: Negative
		Positive	Negative

Figure 8: Confusion matrix for binary classification. Green cells = correct predictions; Red cells = errors. TP and TN are correct; FP and FN are mistakes.

- **TP** (True Positive): Predicted Positive, Actually Positive. ✓
- **TN** (True Negative): Predicted Negative, Actually Negative. ✓
- **FP** (False Positive): Predicted Positive, Actually Negative. (Type I error)
- **FN** (False Negative): Predicted Negative, Actually Positive. (Type II error)

6.2 Key Metrics

6.2.1 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The fraction of all predictions that were correct.

Note

Accuracy is misleading when classes are imbalanced. If 95% of emails are not-spam, a model that always predicts “not-spam” gets 95% accuracy but is useless!

6.2.2 Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

“Of all the samples predicted as Positive, how many were actually Positive?”

6.2.3 Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

“Of all the actual Positives, how many did the model correctly find?”

6.2.4 F1-Score

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The **harmonic mean** of Precision and Recall. It is a single balanced metric useful when there is a class imbalance.

Example

A spam filter classifies 10 emails. Results: TP=4, TN=3, FP=2, FN=1.

$$\text{Accuracy} = \frac{4 + 3}{4 + 3 + 2 + 1} = \frac{7}{10} = 0.70$$

$$\text{Precision} = \frac{4}{4 + 2} = 0.67$$

$$\text{Recall} = \frac{4}{4 + 1} = 0.80$$

$$F_1 = 2 \times \frac{0.67 \times 0.80}{0.67 + 0.80} = 0.73$$

6.3 ROC Curve and AUC**Definition**

The **ROC curve** (Receiver Operating Characteristic) plots the *True Positive Rate* (Recall) vs the *False Positive Rate* at different classification thresholds.

$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + TN}$$

AUC (Area Under the ROC Curve) summarises the ROC curve in a single number. AUC = 1.0 is perfect; AUC = 0.5 is random guessing.

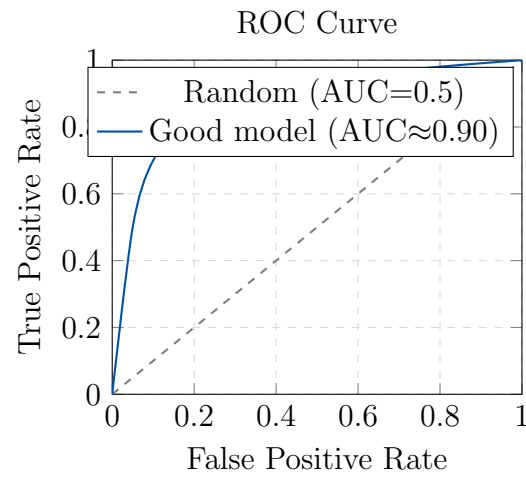


Figure 9: An ROC curve. The closer the curve is to the top-left corner, the better the model.

7 Evaluation Metrics for Regression

In regression, we predict a *continuous* number. We measure how far the predictions are from the true values.

7.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Average of the absolute differences between actual (y_i) and predicted ($\hat{y}_i$) values. Easy to interpret (same units as the target).

7.2 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Squares the errors before averaging. Penalises **large errors** more heavily than MAE.

7.3 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Square root of MSE. Same units as the target – easier to interpret than MSE.

7.4 R-Squared (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of actual values.

R^2 tells us the **proportion of variance** in the target that the model explains. $R^2 = 1$ is perfect; $R^2 = 0$ means the model is no better than predicting the mean.

Example

A model predicts house prices (in thousands of BDT):

Sample	Actual y	Predicted $\hat{y}$	Error $ y - \hat{y} $
1	50	48	2
2	70	73	3
3	60	58	2
4	80	84	4
5	55	53	2

$$\text{MAE} = \frac{2 + 3 + 2 + 4 + 2}{5} = 2.6 \text{ (thousand BDT)}$$

$$\text{MSE} = \frac{4 + 9 + 4 + 16 + 4}{5} = 7.4$$

$$\text{RMSE} = \sqrt{7.4} \approx 2.72 \text{ (thousand BDT)}$$

Metric	Sensitive to Outliers?	Units
MAE	No	Same as target
MSE	Yes	Squared units
RMSE	Yes	Same as target
R^2	Yes	Unitless (0 to 1)

8 Regularization: Motivation and Intuition

8.1 The Overfitting Problem Revisited

When a model overfits, it has very **large weight values**. Large weights allow the model to create very sharp curves that fit every training point exactly.

Key Insight

Regularization is a technique that *penalises large weights* during training. This forces the model to be simpler and reduces overfitting.

8.2 Intuition: Penalty on Complexity

Without regularization, training minimises only the loss:

$$\text{Minimise } \mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

With regularization, we add a **penalty term** for large weights:

$$\text{Minimise } \mathcal{L}(\mathbf{w}) + \lambda \cdot \text{Penalty}(\mathbf{w})$$

The parameter $\lambda \geq 0$ controls how much we care about the penalty:

- $\lambda = 0$: No regularization (may overfit).
- Large λ : Strongly penalise large weights (may underfit).
- A well-chosen λ : Balances fitting and simplicity.

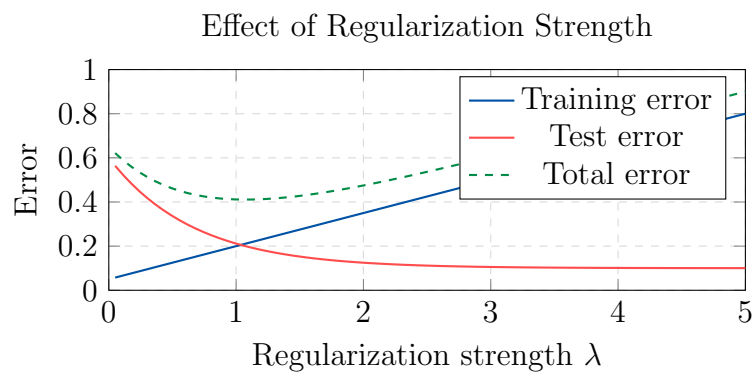


Figure 10: Increasing λ reduces overfitting but may increase underfitting. The optimal λ minimises the test error.

9 L2 Regularization (Ridge Regression)

Definition

L2 Regularization (also called **Ridge Regression** or **Weight Decay**) adds the *sum of squares* of all weights to the loss function:

$$\mathcal{L}_{\text{Ridge}}(\mathbf{w}) = \underbrace{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{original loss}} + \underbrace{\lambda \sum_{j=1}^p w_j^2}_{\text{L2 penalty}}$$

9.1 How It Works

The L2 penalty $\lambda \sum w_j^2$ grows rapidly as weights get large. To minimise the total cost, the optimiser is forced to keep weights small.

Effect on weights:

- All weights are **shrunk towards zero** but *never exactly zero*.
- The model keeps all features but with smaller coefficients.

Example

Simple linear model: $\hat{y} = w_1 x_1 + w_2 x_2 + b$

Without L2: training might give $w_1 = 10$, $w_2 = -8$.

With L2 ($\lambda = 0.5$): training gives $w_1 = 3$, $w_2 = -2$.

The weights are smaller, making the model smoother.

9.2 Geometric Interpretation

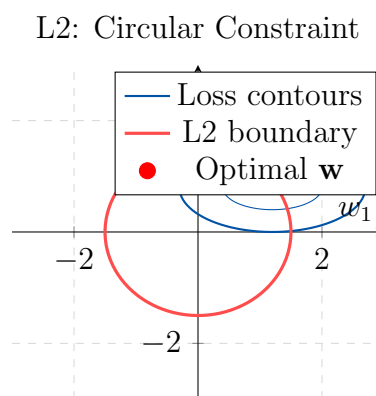


Figure 11: L2 regularization constrains weights to a *circle*. The solution lands on the circle but rarely at an axis (so weights are small but not zero).

9.3 Closed-Form Solution

For linear regression with L2, the solution has a closed form:

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

The term $\lambda \mathbf{I}$ makes the matrix invertible even when features are correlated (multicollinearity problem in plain linear regression).

10 L1 Regularization (Lasso)

Definition

L1 Regularization (also called **Lasso** – Least Absolute Shrinkage and Selection Operator) adds the *sum of absolute values* of all weights to the loss:

$$\mathcal{L}_{\text{Lasso}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |w_j|$$

10.1 Key Property: Sparsity

The most important feature of L1 is that it can set weights **exactly to zero**. This means L1 performs **automatic feature selection**: irrelevant features get weight 0 and are effectively removed from the model.

Example

A model predicts house price from 10 features (area, rooms, age, colour, owner's name, ...).

After L1 regularization, 6 weights become exactly 0.

Remaining non-zero weights: area, rooms, age, location.

L1 automatically identified the 4 truly useful features!

10.2 Geometric Interpretation

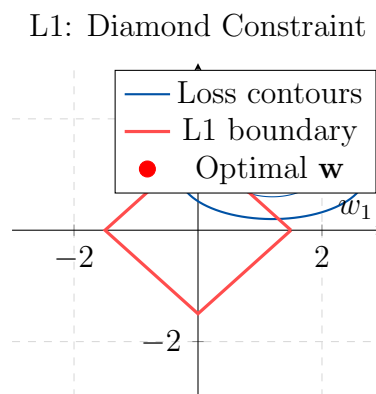


Figure 12: L1 regularization constrains weights to a *diamond* shape. The loss contour often touches a *corner* of the diamond, making one weight exactly zero.

10.3 L1 vs L2 Comparison

Property	L2 (Ridge)	L1 (Lasso)
Penalty term	$\sum w_j^2$	$\sum w_j $
Weights become zero?	No (only small)	Yes (sparse)
Feature selection?	No	Yes
Geometry	Circle	Diamond
Best when	All features matter	Many irrelevant features

Note

Elastic Net combines L1 and L2: $\lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$. It gets the benefits of both.

11 Early Stopping

Definition

Early Stopping is a regularization technique for iterative training (e.g. neural networks, gradient boosting). We stop training before the model starts to overfit, using the validation loss as a signal.

11.1 The Key Idea

During training:

- Training loss *always decreases* as we train more.
- Validation loss *decreases at first* but then *starts increasing* when the model begins overfitting.
- We stop training at the **minimum validation loss**.

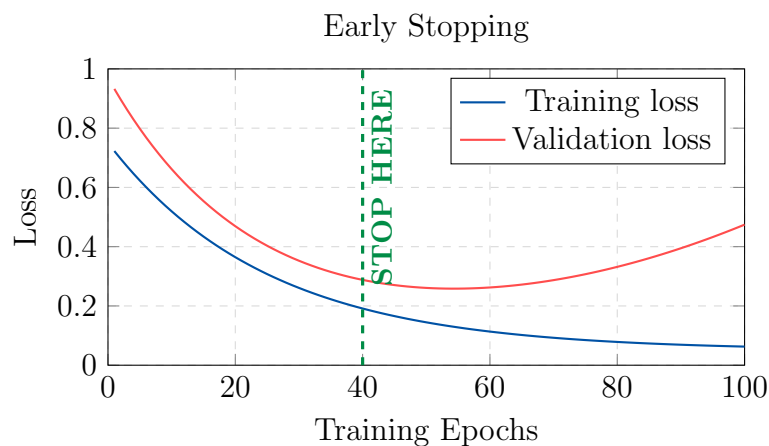


Figure 13: Early stopping: halt training at the epoch where validation loss is minimum.

11.2 Algorithm

1. Split data into training and validation sets.
2. Train the model and monitor validation loss after each epoch.
3. If validation loss has not improved for p consecutive epochs (“patience”), stop training.
4. Restore the model weights from the epoch with the best validation loss.

Example

We train a neural network. Best validation loss occurs at epoch 40. After epoch 40, validation loss increases. With patience = 10, we stop at epoch 50 and use the weights from epoch 40.

Key Insight

Early stopping is **free** – it does not change the model architecture or add extra penalty terms. It is one of the most practical regularization methods.

12 Ensemble Methods Overview

Definition

Ensemble methods combine *multiple models* to produce a prediction that is better than any single model.

12.1 Why Combine Models?

A single model may make mistakes. If different models make *different* mistakes, combining them can cancel out errors.

Example

Three friends predict whether it will rain:

- Friend A (based on clouds): “Yes”
- Friend B (based on humidity): “Yes”
- Friend C (based on wind): “No”

Majority vote: 2 vs 1 $\Rightarrow$ “Yes”.

The combined prediction is usually more reliable than any single opinion.

12.2 Main Ensemble Strategies

Strategy	Models trained	Combined how	Example
Bagging	In parallel	Average / Vote	Random Forest
Boosting	Sequentially	Weighted sum	AdaBoost, XGBoost
Stacking	Independently	Meta-learner	Blended models

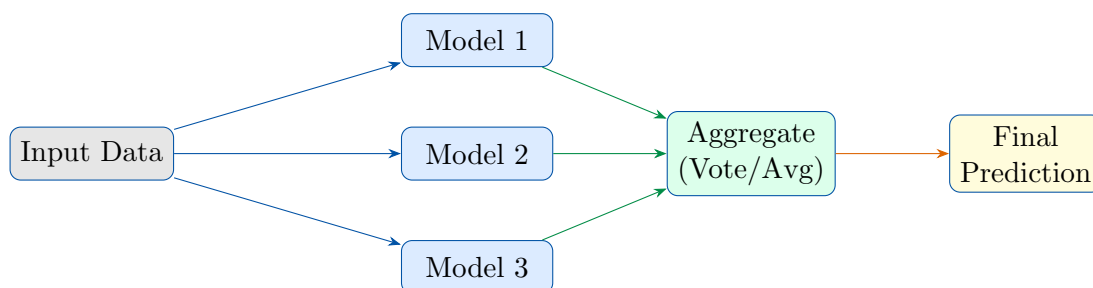


Figure 14: General structure of an ensemble method.

13 Bagging and Random Forests

13.1 Bagging (Bootstrap Aggregating)

Definition

Bagging trains T models *in parallel*, each on a different **bootstrap sample** of the training data. Final prediction is:

- **Classification:** majority vote.
- **Regression:** average of all predictions.

Steps:

1. For $t = 1$ to T :
 - (a) Draw a bootstrap sample D_t (sample with replacement from training data).
 - (b) Train model h_t on D_t .
2. Combine: $\hat{y} = \text{majority_vote}(h_1(x), h_2(x), \dots, h_T(x))$

Key Insight

Bagging **reduces variance** (the main cause of overfitting) while keeping bias roughly the same.

13.2 Random Forests

Definition

A **Random Forest** is Bagging applied to **Decision Trees**, with one extra twist: at each split in each tree, only a *random subset* of features is considered (not all features). This makes the trees *diverse* (less correlated), which improves the ensemble.

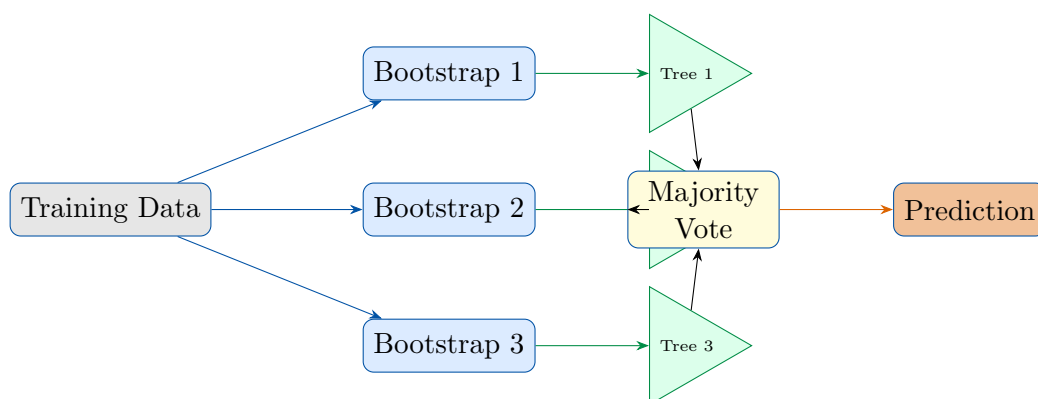


Figure 15: Random Forest: multiple decision trees trained on different bootstrap samples, combined by majority voting.

13.3 Key Hyperparameters of Random Forests

Parameter	Meaning	Typical value
T (n_estimators)	Number of trees	100 – 500
max_features	Features per split	$\sqrt{p}$ (classif.), $p/3$ (regress.)
max_depth	Maximum tree depth	None (full) or 5–20
min_samples_leaf	Min samples per leaf	1–5

Example

We build a Random Forest to classify whether a bank loan will default. Features: income, age, debt, employment years, credit score (5 features). At each split, only $\sqrt{5} \approx 2$ random features are considered. This prevents any single dominant feature from appearing in every tree, making the trees more diverse.

14 Boosting Methods

Definition

Boosting trains models *sequentially*. Each new model focuses on the **errors** (misclassified or poorly predicted examples) made by the previous models. The final prediction is a **weighted combination** of all models.

14.1 AdaBoost (Adaptive Boosting)

AdaBoost is one of the earliest and most well-known boosting algorithms.

Algorithm:

1. Assign equal weight $w_i = \frac{1}{n}$ to every training sample.
2. **For** $t = 1$ to T :
 - (a) Train a *weak classifier* h_t (e.g. a decision stump) on the *weighted* data.
 - (b) Compute the weighted error: $\epsilon_t = \sum_{i:h_t(x_i) \neq y_i} w_i$
 - (c) Compute model weight: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$
 - (d) **Increase** the weight of misclassified samples.
 - (e) **Decrease** the weight of correctly classified samples.
 - (f) Normalise weights.
3. Final prediction: $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$

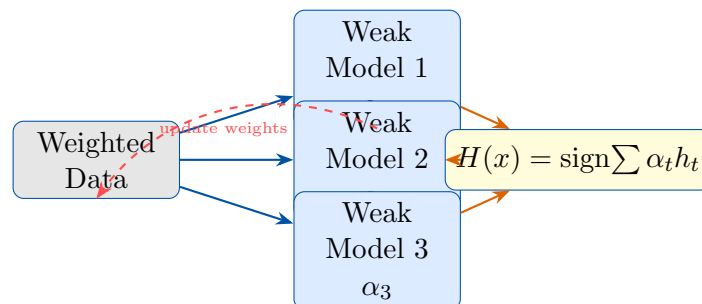


Figure 16: AdaBoost: each weak model is assigned a weight α_t based on its accuracy. Misclassified samples receive higher weights in the next round.

Example

Classifying emails as spam/not-spam with 10 examples.

- **Round 1:** All equal weights. Stump 1 misclassifies 2 emails. $\Rightarrow$ Those 2 emails get higher weight.
- **Round 2:** Stump 2 focuses on the 2 hard emails. Misclassifies 1 different email. $\Rightarrow$ That email gets higher weight.

- **Round 3:** Stump 3 focuses on the remaining hard example.
- **Final:** weighted vote of all 3 stumps.

14.2 Gradient Boosting

Definition

Gradient Boosting generalises boosting by fitting each new model to the *residual errors* (gradients of the loss) of the current ensemble. The ensemble is built by gradient descent in function space.

Key idea:

$$F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$$

where:

- $F_{t-1}(x)$ is the current ensemble prediction.
- $h_t(x)$ is a new tree trained on the **negative gradient** (residuals).
- η is the learning rate (step size).

14.3 XGBoost

Definition

XGBoost (Extreme Gradient Boosting) is an optimised, regularised version of gradient boosting. It is one of the most powerful algorithms for structured/tabular data and has won many Kaggle competitions.

Improvements over standard gradient boosting:

- Adds L1 and L2 regularization on tree weights to prevent overfitting.
- Uses second-order (Hessian) information for better updates.
- Handles missing values automatically.
- Supports parallel computation (faster training).

14.4 Boosting vs Bagging Comparison

Property	Bagging (Random Forest)	Boosting (AdaBoost, XGBoost)
Training order	Parallel	Sequential
Focus	Reduces variance	Reduces bias
Overfitting risk	Low	Higher (needs tuning)
Speed	Fast (parallelisable)	Slower
Typical performance	Good baseline	Often best on tabular data
Interpretability	Moderate	Low

Key Insight

- Use **Random Forest** when you need a fast, robust baseline with low overfitting risk.
- Use **XGBoost / LightGBM** when you want the highest accuracy on structured data and have time to tune hyperparameters.

Chapter Summary

Topic	Key Takeaway
Overfitting	Model is too complex; memorises training data; bad on new data.
Underfitting	Model is too simple; misses patterns even in training data.
Model Evaluation	Always evaluate on unseen test data; training score is not enough.
Dataset Splitting	Use train/validation/test or holdout; never leak test data into training.
Cross-Validation	K-Fold uses data more efficiently; use stratified K-Fold for imbalanced classes.
Bootstrap	Sampling with replacement; $\approx 36.8\%$ data is out-of-bag for testing.
Classification Metrics	Accuracy, Precision, Recall, F1, AUC-ROC. Use F1/AUC for imbalanced data.
Regression Metrics	MAE (robust), MSE/RMSE (penalises large errors), R^2 (variance explained).
Regularization	Adds penalty on weights to prevent overfitting. Controls complexity via λ .
L2 (Ridge)	Shrinks weights to near zero; keeps all features; good for correlated features.
L1 (Lasso)	Sets some weights to exactly zero; performs feature selection.
Early Stopping	Stop training when validation loss stops improving; prevents overfitting for free.
Ensemble Methods	Combine multiple models; reduces errors through diversity.
Bagging / RF	Parallel training on bootstrap samples; reduces variance; robust and fast.
Boosting	Sequential training on errors; reduces bias; powerful but needs tuning.